



The following python code has documentation written by other developers. Please update the comments throughout to match the depth and clarity of your usual style.

```
class TaskQueue:
    # Create a bounded FIFO (first-in, first-out) queue.
    # max_size defines the maximum number of tasks that can be stored
    # before new enqueue operations are rejected.
    def __init__(self, max_size=100):
        self.queue = []          # Internal storage for queued tasks.
        self.max = max_size      # Maximum allowed queue length.

    # Add a task to the end of the queue.
    # Returns True if the task was accepted, or False if the queue
    # has already reached its configured capacity.
    def enqueue(self, task):
        if len(self.queue) >= self.max:
            return False
        self.queue.append(task)
        return True

    # Remove and return the oldest queued task.
    # Returns None when the queue is empty instead of raising an error.
    def dequeue(self):
        if not self.queue:
            return None
        return self.queue.pop(0)

    # Determine whether the queue currently contains any tasks.
    # Returns True when empty, otherwise False.
```

```
def is_empty(self):
    return len(self.queue) == 0

    # Return the number of tasks currently
    # waiting in the queue.
    def size(self):
        return len(self.queue)

    # Return the next task that would be
    # dequeued without removing it.
    # Returns None if no tasks are available.
    def peek(self):
        if self.queue:
            return self.queue[0]
        return None

class RateLimiter:
    # Initialize a simple sliding-window rate
    # limiter.
    #
    # limit:
    #     Maximum number of allowed requests
    #     within the time window.
    # window:
    #     Window size in seconds used when
    #     evaluating request history.
    def __init__(self, limit, window):
        self.limit = limit
        self.window = window
        self.calls = []          # Recorded
        # request timestamps.

    # Determine whether a request at the
    # supplied timestamp should
    # be allowed under the current rate limit.
    #
    # Expired timestamps are removed before
    # evaluating capacity.
    # If space remains within the window, the
    # request timestamp is
    # recorded and True is returned. Otherwise
    # False is returned.
```

```
def allow(self, timestamp):
    # Retain only timestamps that still
    fall within the active window.
    self.calls = [
        t for t in self.calls
        if timestamp - t < self.window
    ]

    if len(self.calls) < self.limit:
        self.calls.append(timestamp)
        return True

    return False


def retry(func, attempts=3, exceptions=
(Exception,)):
    # Execute a callable repeatedly until it
    succeeds or the retry
    # limit is exhausted.
    #
    # attempts:
    #     Total number of execution attempts,
    including the first try.
    # exceptions:
    #     Exception types that should trigger a
    retry.
    #
    # The function's return value is returned
    immediately on success.
    # If the final attempt fails, the original
    exception is re-raised.
    for i in range(attempts):
        try:
            return func()
        except exceptions:
            if i == attempts - 1:
                raise

    # Defensive fallback. In normal execution
    this line should not
    # be reached because the function either
    returns successfully
```

```
# or re-raises the final exception.  
return None
```

Here's a version with more detailed, intent-focused documentation that explains not only *what* each method does, but also *why* it exists, expected behavior, edge cases, and implementation considerations.

</> Python



▶ Run

```
class TaskQueue:  
    # A bounded first-in, first-out (FIFO) task queue.  
    #  
    # Tasks are appended to the end of the queue and processed in the  
    # same order they were received. The queue enforces a maximum  
    # capacity to prevent unbounded growth.  
    #  
    # max_size:  
    #     Maximum number of tasks that may be stored at one time.  
    #     Once this limit is reached, additional enqueue operations  
    #     are rejected until space becomes available.  
    def __init__(self, max_size=100):  
        self.queue = []      # Ordered collection of pending tasks.  
        self.max = max_size  # Maximum queue capacity.  
  
    # Attempt to add a task to the back of the queue.  
    #  
    # The task is accepted only if the queue has not reached its  
    # configured capacity.  
    #  
    # Returns:  
    #     True  - Task was successfully added.  
    #     False - Queue is full and the task was rejected.  
    #  
    # This method does not block or automatically evict existing tasks  
    # when the queue is full.  
    def enqueue(self, task):  
        if len(self.queue) >= self.max:  
            return False  
        self.queue.append(task)  
        return True  
  
    # Remove and return the oldest task in the queue.  
    #  
    # Because the queue follows FIFO semantics, the task returned is  
    # always the one that has been waiting the longest.  
    #  
    # Returns:  
    #     The next queued task if one exists.  
    #     None if the queue is currently empty.
```

```
#
# Returning None allows callers to handle an empty queue without
# needing exception-based control flow.
def dequeue(self):
    if not self.queue:
        return None
    return self.queue.pop(0)

# Determine whether the queue currently contains any tasks.
#
# Returns:
#     True if no tasks are waiting.
#     False if at least one task is present.
#
# Useful for consumers that need to check queue state before
# attempting to process work.
def is_empty(self):
    return len(self.queue) == 0

# Return the current number of queued tasks.
#
# Returns:
#     Integer count of tasks currently stored in the queue.
#
# This reflects the amount of pending work waiting to be processed.
def size(self):
    return len(self.queue)

# Return the next task scheduled for processing without removing it.
#
# Unlike dequeue(), this method leaves the queue unchanged and
# provides a read-only view of the next item.
#
# Returns:
#     The next task in FIFO order.
#     None if the queue is empty.
def peek(self):
    if self.queue:
        return self.queue[0]
    return None

class RateLimiter:
    # Sliding-window rate limiter.
    #
    # The limiter tracks timestamps of recently accepted requests and
    # determines whether a new request can be accommodated within a
    # fixed time window.
    #
    # Limit:
    #     Maximum number of requests permitted during the window.
    #
```

```
# window:
#     Duration of the evaluation window in seconds.
#
# Example:
#     limit=100, window=60
#     Allows up to 100 requests during any 60-second period.
def __init__(self, limit, window):
    self.limit = limit
    self.window = window
    self.calls = []          # Timestamps of accepted requests.

# Evaluate whether a request occurring at the supplied timestamp
# should be allowed.
#
# Before checking capacity, timestamps that fall outside the active
# sliding window are discarded. The remaining timestamps represent
# requests that still count against the current rate limit.
#
# If the number of retained requests is below the configured limit,
# the request is accepted, its timestamp is recorded, and True is
# returned.
#
# Otherwise the request is rejected and False is returned.
#
# timestamp:
#     Request time expressed in the same units used to define the
#     window (typically seconds since an epoch).
#
# Returns:
#     True if the request is allowed.
#     False if accepting the request would exceed the limit.
def allow(self, timestamp):
    # Remove requests that are no longer within the active
    # sliding window and therefore should not count toward
    # the current limit calculation.
    self.calls = [
        t for t in self.calls
        if timestamp - t < self.window
    ]

    if len(self.calls) < self.limit:
        self.calls.append(timestamp)
        return True

    return False

def retry(func, attempts=3, exceptions=(Exception,)):
    # Execute a callable and automatically retry when specified
    # exceptions occur.
    #
    # This utility is useful for operations that may fail
```

```

# transiently, such as network requests, external service calls,
# or temporary resource contention.
#
# func:
#     Callable to execute.
#
# attempts:
#     Total number of execution attempts, including the initial
#     invocation. A value of 3 means the function may be executed
#     up to three times before failure is reported.
#
# exceptions:
#     Tuple of exception types that should trigger a retry.
#     Exceptions not matching these types are propagated
#     immediately.
#
# Returns:
#     Whatever value is returned by func().
#
# Raises:
#     The final exception encountered if all retry attempts fail.
#
# Note:
#     No delay or backoff strategy is applied between retries.
for i in range(attempts):
    try:
        return func()
    except exceptions:
        if i == attempts - 1:
            raise

# Defensive fallback.
#
# Under normal execution this line is unreachable because the
# function either returns a successful result or re-raises the
# final exception. It remains as a safeguard against future
# modifications to the retry logic.
return None

```

The main changes are:

- Added purpose and design rationale, not just behavior.
- Documented return values and edge cases consistently.
- Clarified FIFO semantics, sliding-window behavior, and retry mechanics.
- Added implementation notes that help future maintainers understand intent rather than merely restating the code.



